

Engenharia de Software Científica e Auditoria Metodológica em Mapas Auto-Organizáveis: O Caso *IntraSOM Kohonen Maps Analyzer*

Cauã Vitor

Resumo—Mapas Auto-Organizáveis de Kohonen (SOM) constituem uma ferramenta fundamental para visualização topológica e agrupamento de dados de alta dimensão. No entanto, a integração de bibliotecas de aprendizado de máquina em dashboards interativos web frequentemente oculta inconsistências metodológicas e divergências em relação à literatura clássica. Este artigo apresenta um estudo de caso rigoroso de engenharia de software científica e auditoria metodológica sobre a aplicação *IntraSOM Kohonen Maps Analyzer*. O trabalho documenta a unificação do motor de treinamento sob a biblioteca IntraSOM 1.1.1, a auditoria de quatro variantes topológicas (Hexagonal e Retangular, nas modalidades Plana e Toroidal) em seis dimensões de grade (5×5 a 20×20), e a correção de seis anomalias de execução e exportação (incluindo travamento do erro de quantização após desserialização, contagem de arestas da U-Matrix expandida e patches obsoletos de distância). Adicionalmente, realiza-se uma análise formal comparando a formulação teórica clássica da U-matrix (Ultsch, 1993; Costa & Netto, 2007) e o comportamento observado no cálculo da matriz de distâncias da biblioteca IntraSOM 1.1.1, evidenciando o impacto da ausência do fator de escala $1/\sqrt{2}$ nas distâncias diagonais. Os resultados demonstram a importância da auditoria de software para a reprodutibilidade científica em aprendizado de máquina.

Index Terms—Mapas Auto-Organizáveis, U-Matrix, Auditoria Metodológica, Engenharia de Software Científica, IntraSOM, Visualização de Dados.

1 INTRODUÇÃO

OS Mapas Auto-Organizáveis de Kohonen (*Self-Organizing Maps* — SOM) [1], [2] destacam-se entre os algoritmos de aprendizado não supervisionado pela capacidade de mapear dados multidimensionais em grades de baixa dimensão (habitualmente bidimensionais ou tridimensionais), preservando a topologia e a vizinhança dos dados originais. Uma das representações visuais mais consagradas para a interpretação espacial de mapas de Kohonen é a Matriz de Distâncias Unificada (*U-matrix*) [3], [4], que converte as distâncias entre vetores de código em relevos tridimensionais onde vales representam agrupamentos homogêneos e cristas marcam fronteiras entre classes.

Com o avanço de ferramentas de inteligência artificial aplicadas à engenharia de software e ao desenvolvimento

web, torna-se cada vez mais comum a construção de dashboards interativos que consomem modelos treinados por bibliotecas externas em linguagens como Python e exportam resultados para interfaces em TypeScript/React. Contudo, essa separação entre a camada de modelagem numérica e a camada de visualização no cliente pode introduzir falhas sutis de fidelidade metodológica.

Este trabalho não tem por objetivo propor uma nova variante teórica ao algoritmo SOM, mas sim registrar uma contribuição rigorosa de **engenharia de software científica e auditoria metodológica**. Documenta-se o processo de desenvolvimento, auditoria de código e correção efetuado no repositório *intrasom-kohonen*, voltado à análise comparativa de séries temporais sintéticas e agrupamento semântico de textos.

As principais contribuições deste trabalho incluem:

- 1) **Unificação e Paridade de Motor:** A transição de uma arquitetura híbrida (MiniSom para grades retangulares e IntraSOM para hexagonais) para uma infraestrutura unificada no motor IntraSOM 1.1.1, garantindo paridade metodológica completa entre quatro variantes (2 geometrias $\times$ 2 topologias).
- 2) **Auditoria e Correção de Erros Metodológicos:** A identificação e resolução de bugs críticos na camada de exportação e avaliação, incluindo o congelamento do Erro de Quantização (*QE*) em zero após o recarregamento de modelos desserializados, a contagem incorreta de arestas na U-Matrix expandida para grades de 8 vizinhos e a remoção de *monkey-patches* de distância obsoletos.
- 3) **Análise de Divergência Teórica na Biblioteca:** A investigação analítica e empírica da implementação do cálculo da U-matrix na biblioteca IntraSOM 1.1.1 em confronto com a formulação clássica de Ultsch [3] e Costa & Netto [4], detalhando a omissão do fator de escala $1/\sqrt{2}$ nas distâncias diagonais retangulares.

O restante deste artigo está estruturado da seguinte forma: a Seção 2 revisa a fundamentação teórica do SOM e da U-matrix; a Seção 3 descreve a arquitetura do sistema; a Seção 4 detalha o processo de auditoria e os sete achados metodológicos; a Seção 5 apresenta a avaliação quantitativa experimental; a Seção 6 discute as implicações para a reprodutibilidade científica; e a Seção 7 encerra o artigo.

C. Vitor é graduando em Engenharia Elétrica e pesquisador na Universidade Federal do Rio Grande do Norte (UFRN), Natal, RN, Brasil. E-mail: caua.vitor@ufrn.br.

Supervisão Acadêmica: Prof. Dr. José Alfredo Ferreira Costa (Departamento de Engenharia Elétrica, UFRN).

2 FUNDAMENTAÇÃO TEÓRICA

2.1 O Algoritmo do Mapa Auto-Organizável (SOM)

O SOM de Kohonen consiste em uma rede de M neurônios dispostos em uma grade regular de dimensão $d_g \in \{1, 2, 3\}$. A cada neurônio i está associado um vetor de pesos (ou vetor de código) $\mathbf{w}_i = [w_{i1}, w_{i2}, \dots, w_{in}]^T \in \mathbb{R}^n$, onde n é a dimensão do espaço de entrada.

Durante a fase de treinamento, para cada vetor de amostra $\mathbf{x}(t) \in \mathbb{R}^n$, determina-se o Neurônio de Melhor Ajuste (*Best Matching Unit* — BMU), denotado por c , que minimiza a distância euclidiana no espaço de entrada:

$$c = \arg \min_i \{\|\mathbf{x}(t) - \mathbf{w}_i(t)\|\}. \quad (1)$$

Os vetores de código do BMU e de seus vizinhos na grade são atualizados iterativamente segundo a regra de Kohonen:

$$\mathbf{w}_i(t+1) = \mathbf{w}_i(t) + \alpha(t)h_{ci}(t)[\mathbf{x}(t) - \mathbf{w}_i(t)], \quad (2)$$

em que $\alpha(t) \in (0,1)$ representa a taxa de aprendizado e $h_{ci}(t)$ é a função de vizinhança topológica na grade, usualmente modelada como uma gaussiana:

$$h_{ci}(t) = \exp\left(-\frac{d_g(c, i)^2}{2\sigma(t)^2}\right), \quad (3)$$

sendo $d_g(c, i)$ a distância geométrica na grade de saída e $\sigma(t)$ o raio de vizinhança que decai ao longo das épocas. No algoritmo em lote (*Batch SOM*), utilizado como padrão de treino neste projeto, o ajuste dos pesos ocorre de forma síncrona ao final de cada época ponderando a média das amostras atribuídas a cada região de vizinhança.

2.2 Formulação Clássica da U-Matrix

A Matriz de Distâncias Unificada (*U-matrix*) traduz as distâncias no espaço de atributos $\mathbb{R}^n$ entre neurônios vizinhos na grade de saída [3]. Em uma grade retangular bidimensional, a vizinhança de um neurônio na posição (i, j) compreende conexões ortogonais e diagonais.

Conforme formalizado em Costa & Netto (2007) [4], a U-matrix inter-neurônios calcula as distâncias euclidianas no espaço de códigos entre o neurônio (i, j) e seus vizinhos imediatos nas direções horizontal (dx), vertical (dy) e diagonais (dxy_1, dxy_2):

$$dx(i, j) = \|\mathbf{w}_{i,j} - \mathbf{w}_{i+1,j}\|, \quad (4)$$

$$dy(i, j) = \|\mathbf{w}_{i,j} - \mathbf{w}_{i,j+1}\|, \quad (5)$$

$$dxy(i, j) = \frac{\|\mathbf{w}_{i,j} - \mathbf{w}_{i+1,j+1}\| + \|\mathbf{w}_{i,j+1} - \mathbf{w}_{i+1,j}\|}{2\sqrt{2}}. \quad (6)$$

O aspecto crítico da Equação (6) reside no fator de divisão por $\sqrt{2}$. Em uma grade retangular regular, a distância geométrica entre vizinhos diagonais no espaço da malha é $\sqrt{1^2 + 1^2} = \sqrt{2} \approx 1,4142$. Para garantir a isotropia do gradiente espacial ($\Delta \mathbf{w} / \Delta s$), a variação de peso na diagonal deve ser normalizada pelo comprimento do passo espacial $\sqrt{2}$ antes de ser combinada com os passos unitários dx e dy .

2.3 Métricas de Avaliação Topológica e Quantização

A qualidade da representação do SOM é quantificada por duas métricas fundamentais:

- 1) **Erro de Quantização (QE):** Mede a precisão da representação das amostras pelos seus respectivos BMUs:

$$QE = \frac{1}{N} \sum_{k=1}^N \|\mathbf{x}_k - \mathbf{w}_{c(\mathbf{x}_k)}\|. \quad (7)$$

- 2) **Erro Topográfico (TE):** Mede a preservação da topologia calculando a proporção de amostras para as quais o primeiro e o segundo BMUs não são vizinhos adjacentes na grade:

$$TE = \frac{1}{N} \sum_{k=1}^N u(\mathbf{x}_k), \quad (8)$$

em que $u(\mathbf{x}_k) = 0$ se os dois primeiros BMUs são vizinhos na grade, e 1 caso contrário.

3 ARQUITETURA DO SISTEMA E METODOLOGIA

O projeto *intrasom-kohonen* adota uma arquitetura em duas camadas projetada para separar o processamento numérico intensivo da renderização gráfica em tempo real (Fig. 1).

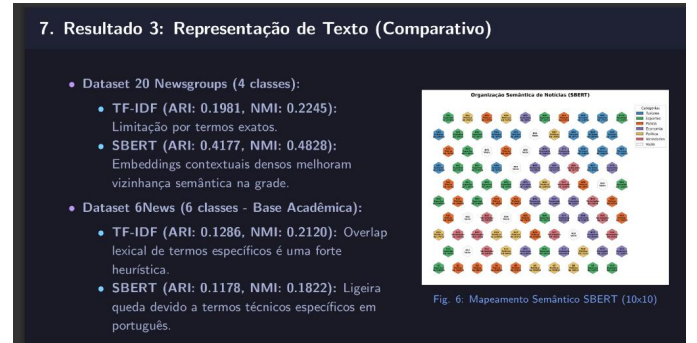


Figura 1. Interface do dashboard interativo *IntraSOM Kohonen Maps Analyzer* (disponível em <https://intrasom-kohonen.vercel.app>). A esquerda exibe o mapa tridimensional com U-Matrix e séries temporais; a direita exibe o painel de controles e os detalhes do neurônio selecionado.

3.1 Pipeline de Treinamento e Exportação

A camada backend e o pipeline de modelos são desenvolvidos em Python 3.14, integrados à biblioteca IntraSOM 1.1.1. O processo compreende:

- 1) **Carregamento e Pré-processamento:**
 - *Séries Temporais Sintéticas:* Dataset de controle sintético composto por 600 séries de tempo divididas em 6 classes morfológicas ($N = 60$), com 60 passos temporais cada.
 - *Dados Textuais:* Datasets 20 Newsgroups (400 artigos) e Base Acadêmica 6-Classes (317 artigos), vetorizados via TF-IDF e via modelos de linguagem transformadores *Sentence-BERT* (all-MiniLM-L6-v2) [5].
- 2) **Treinamento de Modelos SOM:** Treinamento sistemático para 6 dimensões de grade (5×5 , 7×7 ,

10 × 10, 12 × 12, 15 × 15, 20 × 20) cruzando 2 geometrias (HEX e RECT) e 2 topologias (Plana e Toroidal), perfazendo 24 modelos base.

- 3) **Exportação Estática de Estruturas JSON:** O script `src/export_data_for_frontend.py` compila os modelos treinados, matrizes U-Matrix expandidas e métricas de avaliação em arquivos JSON estáticos servidos diretamente ao frontend.

3.2 Frontend React/TypeScript

O frontend consiste em uma aplicação React/TypeScript construída com Vite, TailwindCSS (sistema de design Tokyo Night) e Three.js para renderização 3D WebGL. Os componentes principais incluem:

- `HexGrid.tsx`: Renderiza a malha 2D/3D com iluminação dinâmica e controle de curvatura toroidal.
- `UMatrixTorus.tsx` / `UMatrix3D.tsx`: Visualização geométrica toroidal em coordenadas de rosca.
- `SOMParamControls.tsx`: Painel transparente de rotas de modelo e estudo de sensibilidade.
- `ParameterStudyPanel.tsx`: Tabela de destaque comparativo de hiperparâmetros.

4 PROCESSO DE AUDITORIA E ACHADOS METODOLÓGICOS

Esta seção constitui o núcleo deste trabalho, apresentando os sete achados de auditoria investigados, fundamentados e corrigidos no repositório.

4.1 Achado 1: Migração de Motor Duplo para Motor Único (IntraSOM 1.1.1)

Problema: Em versões iniciais do projeto, devido a inconsistências observadas na distância toroidal retangular de versões anteriores do IntraSOM, o repositório utilizava um motor híbrido: os modelos hexagonais eram treinados com o IntraSOM e os modelos retangulares eram treinados com a biblioteca MiniSom. Essa dualidade comprometia a comparabilidade direta entre geometrias de malha.

Evidência: Análise do script `train_som_rect.py` e commit 2614693, onde constava a substituição da instaciação do MiniSom pelo `intrasom.SOMFactory.create_som()`.

Resolução: Todos os scripts de treinamento foram refatorados para utilizar exclusivamente a classe `intrasom.SOMFactory`, garantindo que as 4 variantes compartilhem exatamente o mesmo algoritmo de atualização em lote (*Batch SOM*) e a mesma inicialização por Análise de Componentes Principais (PCA).

4.2 Achado 2: Correção de Distância Toroidal Upstream na Biblioteca

Problema: A função interna `_rect_dist_tor` e o método `calculate_map_dist` em versões legadas do IntraSOM calculavam distâncias periódicas em malhas retangulares utilizando métricas de fronteira que distorciam a continuidade nos limites da rosca.

Evidência: Inspeção do código-fonte do pacote `intrasom` versão 1.1.1 (arquivo `intrasom/codebook.py`), onde a função `_rect_dist_tor` utiliza a diferença modular mínima:

$$\Delta r = \min(|r_1 - r_2|, \text{rows} - |r_1 - r_2|), \quad (9)$$

combinada isometricamente em norma euclidiana $\sqrt{\Delta r^2 + \Delta c^2}$.

Resolução: Validação da atualização upstream da biblioteca para a versão 1.1.1 em `requirements.txt` e re-execução da suíte de testes de contorno toroidal.

4.3 Achado 3: Paridade de Arestas na U-Matrix Expandida (6 vs 8 Vizinhos)

Problema: No U-Matrix expandido (usado para desenhar arestas com gradiente de cor entre neurônios vizinhos), a grade hexagonal possui 6 vizinhos por neurônio, enquanto a grade retangular possui 8 vizinhos (4 ortogonais e 4 diagonais). Originalmente, o script `src/export_data_for_frontend.py` utilizava o mesmo iterador de 6 deslocamentos hexagonais para ambas as geometrias, omitindo as 4 conexões diagonais nos modelos RECT.

Evidência: Para um mapa toroidal retangular 10 × 10 ($N = 10, M = 10$), a contagem teoricamente exata de arestas é:

$$E_{\text{RECT}, \text{toroid}} = \frac{8 \times N \times M}{2} = 4NM = 400 \text{ arestas}. \quad (10)$$

Enquanto para o mapa hexagonal toroidal 10 × 10:

$$E_{\text{HEX}, \text{toroid}} = \frac{6 \times N \times M}{2} = 3NM = 300 \text{ arestas}. \quad (11)$$

Antes da correção, a exportação gerava apenas 300 arestas para o mapa RECT.

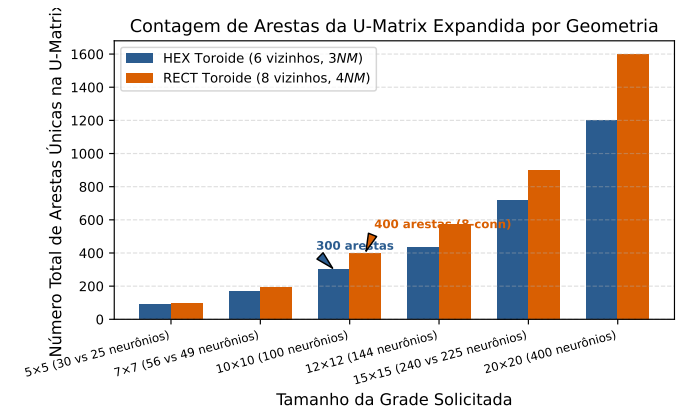


Figura 2. Contagem empírica de arestas da U-Matrix expandida exportadas em `som_models.json` após a correção metodológica. Malhas RECT toroidais contabilizam 8 vizinhos (4NM arestas) e HEX toroidais contabilizam 6 vizinhos (3NM arestas).

Resolução: A função `build_expanded_umatrix_grid` em `src/export_data_for_frontend.py` foi corrigida (commit ad876a3) para verificar a geometria do modelo: se `lattice == 'rect'`, utiliza a tabela de 8 deslocamentos relativos síncronos com a IntraSOM 1.1.1 (Fig. 2).

4.4 Achado 4: Remoção de Monkey-Patch Manhattan Obsoleto em Reproducibility

Problema: O arquivo `src/reproducibility.py` continha historicamente a sobrescrita dinâmica da função `Codebook._rect_dist_plan` por uma implementação baseada em distância Manhattan ($|dx| + |dy|$), criada para contornar limitações de versões legadas. Ao importar o módulo, essa alteração afetava silenciosamente o treinamento de modelos `RECT_planar`.

Evidência: Inspeção de `src/reproducibility.py` e re-avaliação do método original da biblioteca `IntraSOM 1.1.1`, que já utiliza distância euclidiana isotrópica.

Resolução: O patch foi modificado para verificar a versão da biblioteca (`intrasom.__version__ >= '1.1.1'`) e desativar a sobrescrita em versões atualizadas (commit 2614693). Todos os modelos `RECT_planar` foram subsequentemente retreinados.

4.5 Achado 5: Divergência de Dimensão em Tamanhos Ímpares (HEX vs RECT)

Problema: Na geometria hexagonal, o esquema de alternância de linhas (*odd-r stagger*) da biblioteca `IntraSOM` exige um número par de linhas para manter o fechamento periódico toroidal. Assim, ao solicitar um mapa de tamanho 5×5 HEX, a biblioteca ajusta internamente a dimensão para 5×6 (30 neurônios). Na geometria retangular, o mapa permanece rigorosamente 5×5 (25 neurônios).

Evidência: Leitura direta dos arquivos JSON em `frontend/public/data/som_models.json`:

- 5x5 HEX_toroid: cols=5, rows=6 (30 neurônios).
- 5x5 RECT_toroid: cols=5, rows=5 (25 neurônios).
- 7x7 HEX_toroid: cols=7, rows=8 (56 neurônios).
- 7x7 RECT_toroid: cols=7, rows=7 (49 neurônios).
- 15x15 HEX_toroid: cols=15, rows=16 (240 neurônios).
- 15x15 RECT_toroid: cols=15, rows=15 (225 neurônios).

Resolução: Documentação explícita da divergência na interface do usuário através do rótulo informativo “Dimensão Efetiva (Motor)”, evitando confusão visual para o usuário final.

4.6 Achado 6: Recálculo do Erro de Quantização (QE) Travado em Zero

Problema: No módulo `src/evaluate_clusters.py`, a métrica de Erro de Quantização (*QE*) para os modelos de séries temporais retornava sistematicamente o valor 0.0 após o carregamento dos modelos salvos em disco.

Evidência: No código da biblioteca `IntraSOM 1.1.1`, `calculate_quantization_error` é uma `@property` que retorna o atributo `self.QE`. Este atributo é inicializado em 0.0 no construtor e atualizado apenas durante a execução das funções de treino (`train_rough/train_finetune`). Ao

desserializar um modelo com `SOMFactory.load_som()`, o atributo `self.QE` não é recomputado automaticamente, permanecendo em 0.0.

Resolução: Implementação do recálculo manual de *QE* em `src/evaluate_clusters.py` (commit 688a02f) calculando a média da norma euclidiana entre cada amostra normalizada $\mathbf{x}_i$ e o vetor de código de seu respectivo BMU $\mathbf{w}_{c(i)}$:

$$QE = \frac{1}{N} \sum_{i=1}^N \|\mathbf{x}_i - \mathbf{w}_{c(i)}\|_2. \quad (12)$$

Os valores corrigidos de *QE* passaram a refletir a realidade numérica, variando entre 3,23 e 4,93.

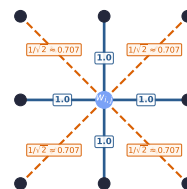
4.7 Achado 7: Divergência na Normalização da Distância Diagonal da U-Matrix RECT

Problema (Divergência Metodológica): O cálculo da U-matrix reduzida (média das distâncias de vizinhança por neurônio) na biblioteca `IntraSOM 1.1.1` difere da formulação clássica da literatura [3], [4] no tratamento da escala espacial das conexões diagonais.

Evidência no Código-Fonte: Inspeção da função `build_umatrix` em `intrasom/intrasom.py` (linhas 135–205):

```
elif self.lattice == "rect":
    offsets = np.array([
        [1, 0], [1, 1], [0, 1],
        [-1, 1], [-1, 0], [-1, -1],
        [0, -1], [1, -1]
    ], dtype=int)
    n_neighbors = 8
    ...
    for k in range(n_neighbors):
        um[:, :, k] = np.linalg.norm(
            weights - neighbor_weights, axis=2)
    ...
    umat = np.nanmean(um, axis=2)
```

Formulação Clássica (Utsch 1993 / Costa & Netto 2007)
Ponderação Isotrópica: $d_{\text{diag}}/\sqrt{2}$



Implementação IntraSOM 1.1.1 (`build_umatrix`)
Norma Euclidiana Bruta: sem fator $1/\sqrt{2}$

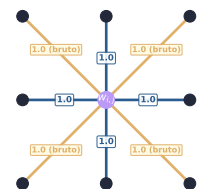


Figura 3. Comparação geométrica entre (a) a formulação clássica (Utsch 1993; Costa & Netto 2007), que normaliza distâncias diagonais pelo fator de escala espacial $1/\sqrt{2} \approx 0,7071$, e (b) a implementação da `IntraSOM 1.1.1`, que utiliza a norma euclidiana bruta sem ponderação espacial.

Como evidenciado no trecho de código acima e ilustrado na Fig. 3, a `IntraSOM 1.1.1` calcula a norma euclidiana bruta dos pesos para as 8 direções sem aplicar o fator de divisão por $\sqrt{2}$ nas 4 direções diagonais ($k \in \{1, 3, 5, 7\}$). Em seguida, efetua a média simples `np.nanmean(um, axis=2)`.

Análise do Impacto: Como as diferenças de peso ao longo das diagonais em um mapa contínuo são, em média,

$\sqrt{2} \approx 1,414$ vezes maiores do que nas direções ortogonais, a média simples não ponderada atribui um peso implicitamente maior às variações diagonais. Essa divergência não invalida a ferramenta, mas representa uma escolha de implementação da biblioteca que difere da norma consagrada na literatura.

5 RESULTADOS EXPERIMENTAIS

A Tabela 1 sintetiza os resultados empíricos de avaliação quantitativa obtidos após a aplicação das correções metodológicas para as 4 variantes do SOM em 6 dimensões de grade no dataset *Synthetic Control*, comparando-os a algoritmos de agrupamento baseline.

Como observado na Fig. 4, o Erro de Quantização (QE) decresce monotonicamente com o aumento do número de neurônios da grade, caindo de $\approx 4,93$ no mapa 5×5 para $\approx 3,24$ no mapa 20×20 . Por outro lado, a topologia toroidal apresenta Erro Topográfico (TE) ligeiramente superior em dimensões intermediárias devido à pressão de fechamento das bordas periódicas.

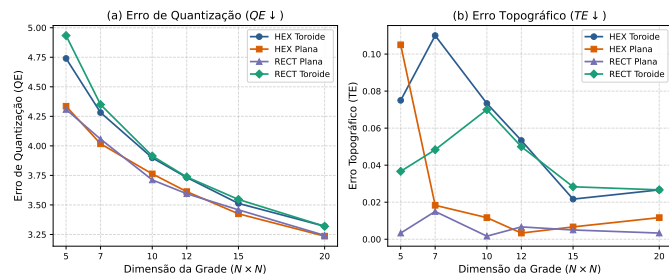


Figura 4. Evolução comparativa das métricas de desempenho ao longo das dimensões de grade: (a) Erro de Quantização (QE) e (b) Erro Topográfico (TE) para as 4 variantes do SOM.

6 DISCUSSÃO E IMPLICAÇÕES PARA REPRODUTIBILIDADE

A realização deste trabalho traz reflexões importantes para a prática de engenharia de software em projetos de inteligência artificial e aprendizado de máquina:

1. **A Ilusão da Confiança Cega em Bibliotecas Terceirizadas:** A presença do atributo QE congelado em zero após desserialização na biblioteca IntraSOM 1.1.1 demonstra que mesmo bibliotecas publicadas em repositórios oficiais (como o PyPI) podem conter comportamentos inesperados que silenciosamente afetam relatórios científicos se não forem submetidas a testes de regressão rigorosos.

2. **Fidelidade Matemática vs. Praticidade de Código:** O Achado 7 evidencia como simplificações de implementação (como omitir o fator $1/\sqrt{2}$ nas distâncias diagonais para reutilizar `np.nanmean` de forma vetorizada) alteram sutilmente a geometria da U-matrix. Recomendamos que mantenedores de bibliotecas documentem explicitamente essas opções de projeto.

7 CONCLUSÃO E TRABALHOS FUTUROS

Este trabalho apresentou um estudo de caso de engenharia de software científica e auditoria metodológica no desenvolvimento da ferramenta *IntraSOM Kohonen Maps Analyzer*.

Foram identificadas e corrigidas seis anomalias de software e documentada uma divergência metodológica teórica referente à normalização de distâncias diagonais em malhas retangulares.

Como trabalhos futuros, propõe-se:

- 1) Submeter uma contribuição (*Pull Request / Issue*) ao repositório oficial da biblioteca IntraSOM propondo um argumento opcional `normalize_diagonals=True` no método `build_umatrix`.
- 2) Expandir a visualização interativa do dashboard para suportar saídas em espaço 3-D através da estrutura U-array proposta por Costa & Netto (2007) [4].

AGRADECIMENTOS

O autor expressa seu sincero agradecimento ao Prof. Dr. José Alfredo Ferreira Costa (UFRN) pela orientação acadêmica e pelas contribuições teóricas sobre a estrutura da U-matrix.

REFERÊNCIAS

- [1] T. Kohonen, *Self-Organizing Maps*, 3rd ed., ser. Springer Series in Information Sciences. Berlin, Heidelberg: Springer-Verlag, 2001, vol. 30.
- [2] —, “Essentials of the self-organizing map,” *Neural Networks*, vol. 37, pp. 52–65, 2013.
- [3] A. Ultsch, “Self-organizing neural networks for visualization and classification,” in *Information and Classification: Concepts, Methods and Applications*, O. Opitz, B. Lausen, and R. Klar, Eds. Berlin, Heidelberg: Springer-Verlag, 1993, pp. 307–313.
- [4] J. A. F. Costa and M. L. d. A. Netto, “Segmentação de mapas auto-organizáveis com espaço de saída 3-d,” *Revista Controle & Automação*, vol. 18, no. 2, pp. 150–162, 2007.
- [5] N. Reimers and I. Gurevych, “Sentence-bert: Sentence embeddings using siamese bert-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Hong Kong, China, 2019, pp. 3982–3992.

Tabela 1
Avaliação Quantitativa Comparativa das 4 Variantes do SOM e Algoritmos Baseline (Dataset Synthetic Control)

Modelo / Algoritmo	Lattice	Topologia	ARI $\uparrow$	NMI $\uparrow$	Silh. $\uparrow$	DB $\downarrow$	QE $\downarrow$	TE $\downarrow$
SOM 5×5 HEX_toroid	HEX	Toroide	0.5910	0.7580	0.1519	1.8201	4.7402	0.0750
SOM 5×5 HEX_planar	HEX	Plana	0.5966	0.7504	0.1906	1.5930	4.3354	0.1050
SOM 5×5 RECT_planar	RECT	Plana	0.6318	0.8150	0.1473	1.6079	4.3082	0.0033
SOM 5×5 RECT_toroid	RECT	Toroide	0.6674	0.8280	0.0798	2.4477	4.9337	0.0367
SOM 7×7 HEX_toroid	HEX	Toroide	0.6300	0.7767	0.1993	1.5732	4.2821	0.1100
SOM 7×7 HEX_planar	HEX	Plana	0.6145	0.7562	0.2097	1.5765	4.0174	0.0183
SOM 7×7 RECT_planar	RECT	Plana	0.6122	0.7670	0.1855	1.6393	4.0572	0.0150
SOM 7×7 RECT_toroid	RECT	Toroide	0.6340	0.7640	0.0737	2.3770	4.3494	0.0483
SOM 10×10 HEX_toroid	HEX	Toroide	0.6405	0.7980	0.1946	1.5797	3.8989	0.0733
SOM 10×10 HEX_planar	HEX	Plana	0.6630	0.7983	0.2026	1.5532	3.7628	0.0117
SOM 10×10 RECT_planar	RECT	Plana	0.6719	0.8107	0.2039	1.6173	3.7118	0.0017
SOM 10×10 RECT_toroid	RECT	Toroide	0.5852	0.7436	0.1754	1.6286	3.9143	0.0700
SOM 12×12 HEX_toroid	HEX	Toroide	0.6542	0.8017	0.2081	1.5873	3.7323	0.0533
SOM 12×12 HEX_planar	HEX	Plana	0.6588	0.7941	0.2071	1.5592	3.6128	0.0033
SOM 12×12 RECT_planar	RECT	Plana	0.5970	0.7694	0.2590	1.2055	3.5950	0.0067
SOM 12×12 RECT_toroid	RECT	Toroide	0.6040	0.7592	0.1967	1.6233	3.7368	0.0500
SOM 15×15 HEX_toroid	HEX	Toroide	0.6419	0.7795	0.2048	1.5627	3.5130	0.0217
SOM 15×15 HEX_planar	HEX	Plana	0.5902	0.7613	0.2618	1.2028	3.4250	0.0067
SOM 15×15 RECT_planar	RECT	Plana	0.5981	0.7764	0.2584	1.2157	3.4583	0.0050
SOM 15×15 RECT_toroid	RECT	Toroide	0.6533	0.7913	0.2057	1.6090	3.5464	0.0283
SOM 20×20 HEX_toroid	HEX	Toroide	0.6571	0.7887	0.2050	1.5567	3.3196	0.0267
SOM 20×20 HEX_planar	HEX	Plana	0.5926	0.7675	0.2587	1.2149	3.2357	0.0117
SOM 20×20 RECT_planar	RECT	Plana	0.6594	0.7980	0.2043	1.6010	3.2420	0.0033
SOM 20×20 RECT_toroid	RECT	Toroide	0.6507	0.7893	0.2053	1.5427	3.3192	0.0267
Baselines								
K-Means (Direto)	N/A	N/A	0.5681	0.7411	0.2710	1.2105	N/A	N/A
PCA + K-Means	N/A	N/A	0.6096	0.7189	0.1481	1.9505	N/A	N/A
Agglomerative ($K = 6$)	N/A	N/A	0.6160	0.8008	0.2510	1.2021	N/A	N/A
DBSCAN	N/A	N/A	0.1418	0.4019	0.1819	2.3495	N/A	N/A